CampusCart (AcroCart) — Project Explanation Guide

Created for Minor Project Exhibition (6th Semester)
Acropolis Institute of Technology & Research, Indore
Designed for Beginners & Presentation Evaluation

1. Project Overview & The Real-World Problem

CampusCart (also known as AcroCart) is a peer-to-peer digital marketplace built specifically for the college campus. It allows students to buy, sell, and trade second-hand college essentials like textbooks, reference materials, lab coats, drafters, scientific calculators, and other items.

The Problem it Solves (Why we built this):

- **1. Unorganized informal groups:** Usually, students sell books via informal WhatsApp groups. These chats get flooded, messages get lost, and there is no searchable catalog.
- **2. Security & Trust:** Public platforms like OLX or Facebook Marketplace are filled with spam, fake profiles, and require meeting strangers outside of campus. CampusCart solves this by forcing all transactions to occur inside the safe, verified college campus.
- **3. Domain Restricting:** To enter the marketplace, users must log in using their official college email. This ensures that only actual students can buy or sell.

2. The Technology Stack Explained Simply

You do not need to be a coding genius to explain the technology behind this app. Here is a simple breakdown of the technologies used, written in everyday language so you can explain them to your evaluators:

A. React 19 (The Building Blocks)

React is like a box of Lego. Instead of writing one massive, complicated webpage, React allows us to break the interface down into small, reusable Lego blocks called 'Components'. For example, the Navbar (navigation bar) is a single Component, and the Product Card is another Component. If we want to display 10 items, we don't code 10 items manually — we just tell React to reuse the 'Product Card' Component 10 times, pasting in the unique data (title, price) for each.

B. Next.js 16 (The Manager)

Next.js is a framework built on top of React. Think of it as the manager of the website. It handles:
1. Routing: When a user clicks 'Sell', Next.js loads the '/sell' page. When they click 'Profile', it

loads '/profile'.
2. Performance: It packages all the JavaScript and assets so the website loads extremely fast.
3. Static Exporting: It allows us to build the app as a set of static files (HTML, CSS, JS) that can be hosted on a fast, cheap cloud server.

C. CSS Modules (The Scoped Designer)

CSS is what gives the website its styling — the colors, gradients, card shapes, and fonts. Normally, if you write CSS, styles from one page can accidentally leak and mess up other pages. CSS Modules solves this. It makes the styles 'scoped' (locked) to that specific component. So, the styles in 'profile.module.css' only affect the profile page, and cannot break anything on the homepage.

D. Firebase (The Serverless Backend)

Normally, a website requires you to write a separate database server (using Node.js, Django, or Java) and rent database servers. Firebase is a 'Backend-as-a-Service' (BaaS). It is a cloud database and auth system provided by Google. It handles all backend requirements for us automatically, so we don't have to code a backend server from scratch. We use three Firebase features:

- **1. Firebase Authentication:** Handles the Google Sign-in popup. It verifies who the user is and retrieves their profile picture, name, and email address.
- **2. Cloud Firestore (The Database):** A cloud-based database that stores listings, chat messages, and user settings. It stores data in 'documents' (like files inside folders). When a user updates their settings or sends a message, it updates Firestore in real-time.
- **3. Firebase Hosting:** This is the cloud server that hosts our compiled Next.js files on the internet so that anyone can open the live URL: https://campuscart-9c677.web.app.

E. Progressive Web App (PWA)

CampusCart is built as a PWA. This means that when a user opens the website on their mobile phone, their browser shows an 'Install App' button. Once clicked, the website gets installed on their home screen with a custom icon and runs in standalone mode (without the browser address bar), looking and feeling exactly like a native Android or iOS app.

3. How the Key Features Work Under the Hood

Here is the simple, conceptual breakdown of the code logic. If the evaluator asks, 'How did you program X?', these explanations will give you the exact logic to present.

Feature 1: Locking logins to college email (@acropolis.in)

How it's coded: When a student clicks 'Sign in with Google', Firebase Auth triggers a popup. Once they log in, we write an event listener in the file 'src/lib/AuthContext.js'. The code checks if the email address ends with '@acropolis.in'. If yes, it logs them in. If no (e.g. they logged in with their personal gmail), the code automatically logs them out, clears the session, and displays an error saying 'Access restricted to @acropolis.in emails only.'

Feature 2: Real-Time Listings (onSnapshot)

How it's coded: Normally, to get new data on a website, you have to refresh the page. In our app, we use a Firestore feature called 'onSnapshot'. It creates a live, open tunnel between the user's browser and the database. The moment a student uploads a new item to the database from their phone, the database tells all active browsers, 'Hey, here is a new listing!' and the homepage updates automatically in under 200 milliseconds without requiring a page reload.

Feature 3: Live In-App Chat system

How it's coded: When a buyer opens an item details page and clicks 'Chat Now', Next.js redirects them to the Profile page and opens the 'Chats' tab. The code checks if a chat thread already exists in Firestore between the buyer and seller. If yes, it loads that thread. If not, it creates a new chat document. Inside the chat, we use a sub-collection called 'messages'. Every time a user types a message and hits send, it adds a new document to the database. An 'onSnapshot' listener displays these messages on both users' screens instantly, creating a WhatsApp-like real-time chat experience.

Feature 4: Local Wishlist (savedItems)

How it's coded: When a user clicks the Heart icon on a product, the code adds that product's unique ID to an array inside the browser's 'LocalStorage' (a simple database built directly into every web browser). When they visit the 'Saved' tab, the code reads these IDs from LocalStorage and fetches those listings from Firestore. This saves server space and works instantly without needing a backend server table.

4. Database Schema Design (Firestore)

Firestore is a NoSQL database. It organizes data in 'Collections' (folders) which contain 'Documents' (files). Each document contains 'Fields' (data pairs). We have 3 main Collections:

A. listings Collection:

Stores all items posted for sale. Each document in this collection has:
- title (String): Name of the item (e.g., 'Engineering Graphics Drafter')
- description (String): Details about the item's condition
- price (Number): Selling price in Rupees (e.g., 250)
- category (String): e.g., 'books', 'electronics', 'fashion', 'home', 'sports'
- status (String): 'active' (for sale) or 'sold' (sold items)
- sellerId, sellerName, sellerEmail, sellerPhoto: Stored directly so we know who is selling it
- createdAt (Timestamp): When it was posted, to sort items newest first

B. chats Collection:

Stores the active conversation threads. Fields include:
- participants (Array): A list of the two users' IDs involved in the chat
- participantDetails (Map): Profile names and pictures of the participants

• itemName (String): The item they are negotiating about
• lastMessage (String): Snippet of the last text sent, shown in the sidebar
• Sub-collection 'messages': Contains the actual text bubbles with 'senderId', 'text', and 'createdAt'.

C. users Collection:

Stores individual student settings. Fields include:
• phone (String): Their WhatsApp/phone number for contact hooks
• department (String): e.g., 'CSE', 'IT', 'EC' (used to match classmates)
• semester (String): e.g., '6' (their current semester level)
• matchAlerts (Boolean): True/False if they want department match highlights
• blockedUsers (Array): List of user IDs they have muted/blocked

5. Project Exhibition Viva Q&A (Be Prepared!)

Here are the most common questions an evaluator will ask during your project exhibition, along with the exact, easy-to-understand answers you should give:

Q1: What is the main purpose of this project?
Answer: CampusCart is an exclusive, secure peer-to-peer marketplace app designed for college students to trade second-hand items. It replaces unorganized student WhatsApp groups and unsafe open platforms like OLX by locking authentication to our college domain (@acropolis.in) and forcing safe, physical transactions on campus.

Q2: What is the tech stack of this application?
Answer: We used Next.js 16 and React 19 for the frontend (compiled as a Progressive Web App), Vanilla CSS Modules for scoped styling, and Firebase (Authentication, Firestore, Hosting) for the serverless backend.

Q3: Why did you choose Firestore over SQL databases like MySQL?
Answer: Since our app requires real-time messaging/chat and instant listing updates, Firestore's built-in real-time data synchronization ('onSnapshot') was a perfect fit. It allows us to stream database changes directly to the browser without setting up complex WebSocket servers. In addition, Firestore's flexible, document-based structure made development fast and scalable.

Q4: How did you implement domain-locked security?
Answer: We used Firebase Auth's Google login provider. When a user authenticates, the client-side code checks if the returned email address ends with our college's domain name (@acropolis.in). If it doesn't match, we automatically log them out and show a restricted access error. On the database level, we wrote Firestore Security Rules that block read/write operations from unauthenticated users, keeping our database secure.

Q5: What are Firestore Security Rules and why did you update them?
Answer: Firestore Security Rules are configuration scripts deployed on Firebase servers that validate database access requests. They prevent unauthorized attackers from writing or deleting our records. Initially, Firebase sets up temporary development rules which expire after 30 days. We created custom rules (saved in firestore.rules and firebase.json) and deployed them to keep the project open for our academic evaluation while keeping it protected.

Q6: Explain what a Progressive Web App (PWA) is and how it benefits your app.
Answer: A PWA is a hybrid website that can be installed directly onto a mobile phone or desktop as a standalone app, meaning it runs in its own window without the browser address bar. It uses a manifest.json file to provide app icons, branding colors, and locked layout orientations. This gives our users a native mobile app experience without requiring them to download a large app from the Google Play Store or App Store.

Q7: How does local development work without Google Auth blocking localhost?
Answer: Google Sign-In sometimes blocks redirects from localhost in development mode due to referrer restrictions. To solve this, we coded a 'mock login bypass' in our AuthContext.js file. When the app detects that it is running on 'localhost', it automatically logs the session in as a pre-configured mock student ('KAVYANSH RAJPUT'). This allows us to test the entire application interface, settings, and messaging flows locally with ease.

Q8: What is your database structure and how are relationships handled?
Answer: We use a NoSQL database, so relationships are handled using 'denormalization' and 'flat structures'. Each product in the 'listings' collection stores its seller's ID, name, and email directly. Each conversation thread in the 'chats' collection stores an array of the two participants' IDs. This structure makes database reads extremely fast because we do not need to run expensive SQL join queries to fetch seller details or chat history.

Q9: What are the main features of the user dashboard?
Answer: The dashboard (Profile page) provides listing management (mark as sold, edit description, delete listings), wishlist favorites loaded from the browser's LocalStorage, real-time in-app chat windows, and settings where students can update their WhatsApp phone number, department branch, semester, and block/mute users.

Q10: What is the future scope of this project?
Answer: In the future, we plan to implement:
1. Image Upload: Allow users to upload actual photos of their items directly to Firebase Storage (currently it auto-assigns category illustrations).

2. Push Notifications: Integrate Firebase Cloud Messaging (FCM) to alert users about new messages or price drops.
3. UPI Payments: Allow in-app UPI QR codes or transactions for cash-free trading.
4. Multi-Campus Support: Configure the app to support other local colleges by extending the domain-restriction list.